

Trabajo práctico N° 2: UART

Felipe Pillichody; Sol A. Nou

Facultad de Ciencias Exactas, Físicas y Naturales

Arquitectura de computadoras

26 de Octubre, 2025

felipe.pillichody@mi.unc.edu.ar

sol.nou@mi.unc.edu.ar

OBJETIVO

El objetivo del trabajo es implementar un sistema de comunicación UART en FPGA que permita el intercambio de datos con una computadora. El sistema en la Basys3 debe recibir desde la PC una operación y dos operadores, procesarlos en la ALU previamente implementada en el TP1 y devolver el resultado por el mismo canal UART.

Se busca integrar módulos de transmisión y recepción UART con la ALU previamente desarrollada, garantizando la correcta sincronización y formato de los datos transmitidos. Además, se validará la comunicación mediante un script en Python que actúe como interfaz de usuario, enviando y recibiendo los datos para verificar el funcionamiento del sistema completo.

DESARROLLO

El sistema implementado integra una arquitectura modular compuesta por una interfaz UART y una Unidad Aritmética Lógica (ALU), permitiendo la comunicación entre una PC y la FPGA para el procesamiento de operaciones aritméticas y lógicas. La comunicación se establece mediante un enlace serie asincrónico bajo el protocolo UART, donde la PC actúa como maestro del sistema, enviando comandos y datos a la FPGA, que procesa la información y devuelve los resultados. Toda la comunicación se realiza byte a byte, directamente entre los módulos de transmisión, recepción e interfaz principal.

La arquitectura general se organiza jerárquicamente en diferentes módulos, cada uno con una funcionalidad específica. Se compone de dos bloques principales:

- **Subsistema UART:** encargado de la comunicación serie entre la PC y la FPGA.
- **Subsistema ALU:** ejecuta operaciones aritméticas y lógicas sobre dos operandos recibidos vía UART.

Ambos bloques son integrados en el top-level *basys3_uart_alu.v*, que actúa como interfaz entre la placa Basys3 y los módulos internos. Este módulo administra señales, conexión entre *uart_tx.v*, *uart_rx.v* y *uart_baudgen.v* a UART y la ALU, generación del paquete de respuesta y entrega del resultado al transmisor UART.

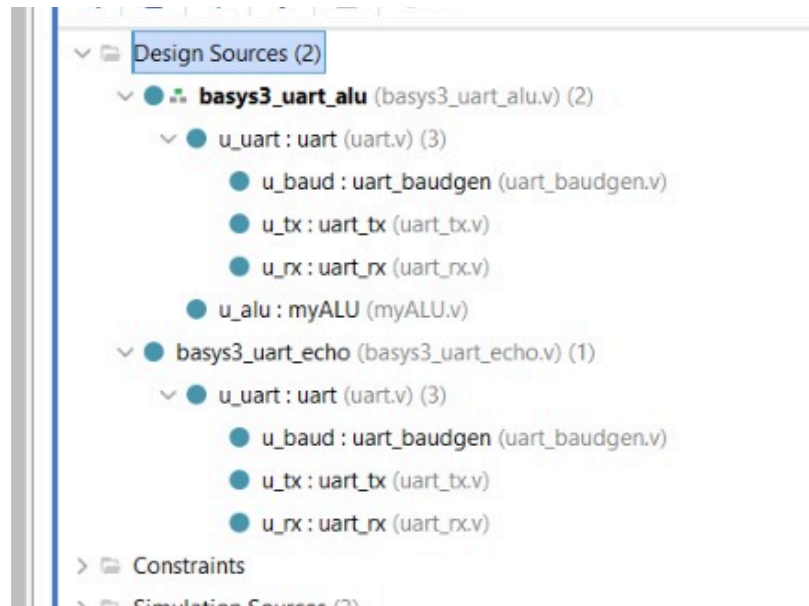


Figura 1: módulos y jerarquía del sistema

Subsistema UART

La implementación del UART se basa en tres módulos fundamentales:

- **uart.v:** módulo top de la comunicación serie. Contiene la lógica de transmisión y recepción de datos, siendo la interfaz simple para TX y RX y abstrae el muestreo. Fue diseñado como una máquina de estados Moore de cuatro estados: IDLE, START, DATA, STOP. Además se considera el sobremuestreo necesario para tener valores precisos. Los módulos que maneja son:

- **uart_tx.v**: implementa el transmisor UART. Se encarga de serializar los datos paralelos provenientes de la FPGA, agregando bits de inicio y parada según el formato estándar del protocolo.
- **uart_rx.v**: implementa el receptor UART. Deserializa los datos recibidos, detecta los bits de inicio y parada, y entrega los datos paralelos al sistema para su posterior procesamiento.
- **uart_baudgen.v**: genera la señal de baud rate de 16 ticks necesaria para sincronizar los módulos de transmisión y recepción, en función de la frecuencia de reloj de la FPGA y la tasa de bits configurada.

SIMULACIÓN Y VALIDACIÓN

Adicionalmente, para asegurar el correcto funcionamiento de cada etapa antes de integrar el sistema completo, se definieron tres fases de trabajo:

Simulación del UART

En primera instancia se buscó validar la comunicación UART exclusivamente dentro del entorno de simulación. Para ello, se implementó un testbench *tb_uart_loopback.v* que interconecta el TX y RX internamente, funcionando como un loopback virtual.

Para observar la trama UART en Vivado, la simulación debe correr al menos 2 ms, ya que la escala temporal del baud rate (115200) genera transiciones de decenas de microsegundos que no se visualizan correctamente en una ventana pequeña.

Validación de comunicación en hardware

Antes de integrar la ALU, se implementó un top dedicado exclusivamente a probar la comunicación serie real entre PC y la FPGA. Para eso se agregó un módulo auxiliar a la FPGA llamado *basys3_uart_echo.v* que permite reenviar los datos tal cual fueron recibidos de la PC, verificando la correcta transmisión y recepción sin involucrar la ALU. Es decir, no aplica ningún procesamiento; funciona como un echo puro.

Luego en la pc, se creó y corrió un script *test_echo.py* de validación del canal UART que envía la cadena Hello Basys3!\r\n y verifica que la respuesta sea idéntica.

Si la cadena coincide byte a byte, queda validado que el baud rate está correctamente configurado en hardware, los niveles lógicos y temporizaciones son compatibles, TX y RX funcionan sin errores y el enlace entre PC y FPGA está operativo.

Una vez validado el modulo UART pasamos a incorporar el modulo ALU.

Comunicación con ALU

En esta etapa se completa el sistema, creando el módulo *basys3_uart_alu.v* que integra: el top uart.v (TX + RX + BaudGen), el top de la ALU (myALU.v), la lógica de interfaz encargada de interpretar paquetes y generar respuestas

La FPGA entonces va a:

1. Recibir los 8 bits (sin FIFO, directamente desde uart_rx)
2. Activar la ejecución de la ALU
3. Calcular el resultado y los flags

4. Preparar la respuesta UART

Complementariamente se realizó un script en Python *uart_alu_pc.py* que actúa como interfaz de usuario. Permite seleccionar una operación, ingresar dos operandos y enviar la información a la FPGA mediante el puerto serie. Luego, recibe y muestra el resultado del procesamiento realizado por la ALU.

CONCLUSION

El sistema desarrollado cumplió el objetivo propuesto de implementar una comunicación UART funcional entre la PC y la FPGA, integrar correctamente los módulos de transmisión, recepción y baud rate con la ALU, y validar el flujo completo de envío, procesamiento y respuesta. La comunicación serie fue verificada mediante simulación, pruebas de *echo* y posteriormente con la ALU, demostrando que el protocolo UART fue entendido, aplicado y que el enlace PC-FPGA funciona de manera confiable. El uso del script en Python permitió además comprobar el funcionamiento del sistema de forma práctica y visual.

Durante el desarrollo se identificaron también mejoras importantes para futuros trabajos. La principal es incorporar una FIFO en transmisión y recepción, ya que el sistema actual opera byte a byte y puede perder datos si el transmisor está ocupado. Además, se observó que el top-level debería estructurarse de forma más modular, evitando concentrar la lógica de interfaz en un solo archivo y facilitando así las pruebas parciales y la reutilización de componentes.